

Java Interview Question Bank — Strong Junior Level

595 interview questions covering Java, Spring, JPA/Hibernate, SQL, testing, microservices, system design, coding, and behavioral scenarios.

Questions only • Ordered Basic → Intermediate → Advanced within each section

Table of Contents

1. Core Java & OOP — 51 questions
2. Collections Framework & Internals — 46 questions
3. Generics — 23 questions
4. Exceptions & Error Handling — 22 questions
5. Java 8+ Features — 36 questions
6. Multithreading & Concurrency — 44 questions
7. JVM Internals, Memory Model & Garbage Collection — 29 questions
8. String Handling & Immutability — 14 questions
9. Design Patterns & Real Usage — 18 questions
10. SOLID Principles & Clean Code — 20 questions
11. Spring & Spring Boot — 43 questions
12. Spring Data JPA & Hibernate — 37 questions
13. REST API Design & Best Practices — 25 questions
14. Databases & SQL — 35 questions
15. Testing — JUnit 5, Mockito & Integration — 29 questions
16. Maven / Gradle, Git & Docker Basics — 32 questions
17. Microservices Fundamentals — 25 questions
18. Entry-Level System Design — 16 questions
19. Coding & Problem-Solving — Live Interview — 28 questions
20. Behavioral & Scenario-Based — 22 questions

Total questions: 595

1. Core Java & OOP

1. [Basic] What is the difference between JDK, JRE, and JVM?
2. [Basic] What are the main features of Java that make it platform independent?
3. [Basic] What is the difference between a primitive type and a reference type?
4. [Basic] What is the difference between stack memory and heap memory at a high level?
5. [Basic] What is the difference between == and equals() in Java?
6. [Basic] What is the contract between equals() and hashCode()?
7. [Basic] What are the four pillars of OOP?
8. [Basic] What is encapsulation, and how do you implement it in Java?
9. [Basic] What is inheritance, and when would you use it?
10. [Basic] What is polymorphism? Give an example involving method overriding.
11. [Basic] What is abstraction, and how is it different from encapsulation?
12. [Basic] What is the difference between an interface and an abstract class?
13. [Basic] Can an abstract class have constructors, fields, and concrete methods?
14. [Basic] Can an interface contain fields and implemented methods in modern Java?
15. [Basic] What is method overloading?
16. [Basic] What is method overriding?
17. [Basic] What rules must be satisfied when overriding a method?
18. [Basic] Can a static method be overridden? What actually happens?
19. [Basic] Can a private method be overridden? Why?
20. [Basic] Can a final method be overridden?
21. [Intermediate] What is the difference between compile-time polymorphism and runtime polymorphism?
22. [Intermediate] Explain upcasting and downcasting. What can go wrong with downcasting?
23. [Intermediate] What is dynamic method dispatch?
24. [Intermediate] Why does Java not support multiple inheritance of classes?
25. [Intermediate] How does Java resolve the diamond problem with default interface methods?
26. [Intermediate] What happens when two interfaces provide the same default method?
27. [Intermediate] What is composition, and why is it often preferred over inheritance?
28. [Intermediate] What is the difference between IS-A and HAS-A relationships?
29. [Intermediate] What does the this keyword mean in different contexts?
30. [Intermediate] What does super do, and when is it required?
31. [Intermediate] What is constructor chaining?
32. [Intermediate] In what order are static fields, instance fields, initializer blocks, and constructors executed?
33. [Intermediate] What happens when a subclass constructor runs before its superclass constructor?
34. [Intermediate] What is a covariant return type?
35. [Intermediate] What is the difference between final, finally, and finalize()?
36. [Intermediate] Why should finalize() not be relied upon for resource cleanup?
37. [Intermediate] What is pass-by-value in Java? Is Java pass-by-reference?

- 38. [Intermediate] What exactly is passed when an object is supplied to a method?
- 39. [Intermediate] What is variable shadowing?
- 40. [Intermediate] What is variable hiding for static fields?
- 41. [Intermediate] What is the difference between an instance method and a static method?
- 42. [Intermediate] When can a static method access instance state?
- 43. [Advanced] What happens if a superclass reference points to a subclass object and both define the same field name and an overridden method?
- 44. [Advanced] Which members participate in runtime polymorphism and which do not?
- 45. [Advanced] What happens if an overridden method throws a broader checked exception than the parent method?
- 46. [Advanced] What happens when a constructor calls an overridable method?
- 47. [Advanced] Why can exposing mutable internal objects break encapsulation even when fields are private?
- 48. [Advanced] How would you design an immutable class with a mutable field?
- 49. [Advanced] What are the consequences of violating the equals/hashCode contract in a HashSet or HashMap?
- 50. [Advanced] What happens if equals() is not symmetric?
- 51. [Advanced] What is the difference between identity equality and logical equality?

2. Collections Framework & Internals

- 52. [Basic] What is the difference between Collection and Collections?
- 53. [Basic] What is the difference between List, Set, Queue, and Map?
- 54. [Basic] When would you choose ArrayList over LinkedList?
- 55. [Basic] What are the main characteristics of HashSet?
- 56. [Basic] What are the main characteristics of TreeSet?
- 57. [Basic] What are the main characteristics of LinkedHashSet?
- 58. [Basic] What is the difference between HashMap, LinkedHashMap, and TreeMap?
- 59. [Basic] Can a HashMap contain a null key and null values?
- 60. [Basic] Can HashSet contain null?
- 61. [Basic] What is the difference between Iterator and ListIterator?
- 62. [Basic] What is ConcurrentModificationException?
- 63. [Intermediate] How does ArrayList grow internally?
- 64. [Intermediate] Why is random access generally fast in ArrayList?
- 65. [Intermediate] Why can inserting into the middle of an ArrayList be expensive?
- 66. [Intermediate] How does LinkedList implement its elements internally?
- 67. [Intermediate] Why is LinkedList often slower than expected in real applications?
- 68. [Intermediate] How does HashMap store entries internally in modern Java?
- 69. [Intermediate] What is a hash collision?
- 70. [Intermediate] How does HashMap handle collisions?
- 71. [Intermediate] What is the purpose of the hashCode() method in HashMap?
- 72. [Intermediate] What happens when two different keys have the same hash code?

73. [Intermediate] What happens if two keys are equal but have different hash codes?
74. [Intermediate] What is the expected time complexity of `get()` and `put()` in `HashMap`?
75. [Intermediate] Under what conditions can `HashMap` performance degrade?
76. [Intermediate] What is load factor in `HashMap`?
77. [Intermediate] What happens when a `HashMap` exceeds its resize threshold?
78. [Intermediate] Why can resizing a `HashMap` be expensive?
79. [Intermediate] Why is a mutable object dangerous as a `HashMap` key?
80. [Intermediate] What is fail-fast iteration?
81. [Intermediate] What is the difference between `Collections.synchronizedMap()` and `ConcurrentHashMap`?
82. [Intermediate] What is the difference between `synchronizedList` and `CopyOnWriteArrayList`?
83. [Intermediate] When is `CopyOnWriteArrayList` appropriate?
84. [Intermediate] What is `BlockingQueue` and where would you use it?
85. [Intermediate] What is the difference between `ArrayDeque` and `Stack`?
86. [Intermediate] What is `PriorityQueue` used for?
87. [Intermediate] What ordering guarantees does `TreeMap` provide?
88. [Advanced] How does `HashMap` use tree bins in modern Java?
89. [Advanced] What conditions cause a `HashMap` bucket to be treeified?
90. [Advanced] Why does `HashMap` use both a treeification threshold and an untreeification threshold?
91. [Advanced] Why is `ConcurrentHashMap` unable to store null keys or null values?
92. [Advanced] How does `ConcurrentHashMap` provide thread-safe access without synchronizing the entire map?
93. [Advanced] What is weakly consistent iteration in `ConcurrentHashMap`?
94. [Advanced] What does `computeIfAbsent()` do, and what mistakes can cause problems when using it?
95. [Advanced] Why can a poorly designed key class cause serious `HashMap` performance problems?
96. [Advanced] What happens if a key's fields used by `hashCode()` change after insertion into a `HashMap`?
97. [Advanced] Compare `synchronizedMap`, `ConcurrentHashMap`, and immutable/unmodifiable maps for concurrent read scenarios.

3. Generics

98. [Basic] Why were generics introduced in Java?
99. [Basic] What is a generic class?
100. [Basic] What is a generic method?
101. [Basic] What is the difference between `List` and `List`?
102. [Basic] What does type safety mean in the context of generics?
103. [Basic] What is a raw type?
104. [Basic] What is the diamond operator?
105. [Intermediate] What is type erasure?
106. [Intermediate] Why can you not create `new T()` directly?
107. [Intermediate] Why can you not create a generic array such as `new T[10]`?

- 108. [Intermediate] What is the difference between `Comparable`, `Comparator`, and `Comparable`?
- 109. [Intermediate] Explain the PECS rule.
- 110. [Intermediate] Why can a `List` be read safely but not normally written to?
- 111. [Intermediate] Why can a `List` accept `Integer` values?
- 112. [Intermediate] What is the difference between bounded type parameters and wildcards?
- 113. [Intermediate] How do generic methods infer their type parameters?
- 114. [Intermediate] What is a recursive type bound such as `>` used for?
- 115. [Advanced] What bridge methods can the compiler generate because of type erasure?
- 116. [Advanced] Why do generics not work directly with primitive types?
- 117. [Advanced] What is heap pollution?
- 118. [Advanced] When is `@SuppressWarnings("unchecked")` potentially justified, and what should surround it?
- 119. [Advanced] Why are generic types invariant in Java?
- 120. [Advanced] Can you overload methods that differ only by their generic parameterization? Why or why not?

4. Exceptions & Error Handling

- 121. [Basic] What is the difference between `Error` and `Exception`?
- 122. [Basic] What is the difference between checked and unchecked exceptions?
- 123. [Basic] What is the purpose of `try-catch-finally`?
- 124. [Basic] What is the difference between `throw` and `throws`?
- 125. [Basic] When would you create a custom exception?
- 126. [Basic] What is exception propagation?
- 127. [Basic] What happens if an exception is not caught?
- 128. [Intermediate] What is `try-with-resources`?
- 129. [Intermediate] Which interface must a resource implement for `try-with-resources`?
- 130. [Intermediate] What are suppressed exceptions?
- 131. [Intermediate] In what order are multiple catch blocks evaluated?
- 132. [Intermediate] Why must a catch block for a subclass exception appear before a catch block for its superclass?
- 133. [Intermediate] Can `finally` fail to execute? Under what circumstances?
- 134. [Intermediate] What happens if both `try` and `finally` return values?
- 135. [Intermediate] Should you catch `Exception` broadly in application code?
- 136. [Intermediate] How should you preserve the original cause when wrapping an exception?
- 137. [Intermediate] What is exception translation in a layered application?
- 138. [Intermediate] When should an exception be logged and when should it simply be propagated?
- 139. [Advanced] What happens to a checked exception thrown from a lambda passed to a standard functional interface?
- 140. [Advanced] How would you design an exception hierarchy for a REST API?
- 141. [Advanced] What is the danger of catching an exception, logging it, and then silently continuing?
- 142. [Advanced] What is the difference between fail-fast error handling and defensive recovery?

5. Java 8+ Features

143. [Basic] What is a lambda expression?
144. [Basic] What is a functional interface?
145. [Basic] What does `@FunctionalInterface` enforce?
146. [Basic] Name common built-in functional interfaces in `java.util.function`.
147. [Basic] What is a method reference?
148. [Basic] What is the difference between a Predicate, Function, Consumer, and Supplier?
149. [Basic] What is a Stream?
150. [Basic] What is the difference between a Collection and a Stream?
151. [Basic] What is the difference between intermediate and terminal stream operations?
152. [Basic] What does `filter()` do?
153. [Basic] What does `map()` do?
154. [Basic] What does `flatMap()` do?
155. [Basic] What does `reduce()` do?
156. [Basic] What does `collect()` do?
157. [Intermediate] Why are streams lazy?
158. [Intermediate] What triggers stream execution?
159. [Intermediate] What is the difference between `map()` and `flatMap()` in a nested collection?
160. [Intermediate] What is the difference between `findFirst()` and `findAny()`?
161. [Intermediate] What is the difference between `anyMatch()`, `allMatch()`, and `noneMatch()`?
162. [Intermediate] What is the difference between `orElse()` and `orElseGet()` in `Optional`?
163. [Intermediate] Why should `Optional` generally not be used as a field in JPA entities?
164. [Intermediate] When should `Optional` be returned from a method?
165. [Intermediate] What is the difference between `map()` and `flatMap()` on `Optional`?
166. [Intermediate] What is the purpose of `peek()`, and why can relying on it for business logic be dangerous?
167. [Intermediate] What are short-circuiting stream operations?
168. [Intermediate] What is the difference between sequential and parallel streams?
169. [Intermediate] When can `parallelStream()` make performance worse?
170. [Intermediate] How does `Collectors.groupingBy()` work?
171. [Intermediate] How would you group objects by one property and count them?
172. [Intermediate] How would you convert a `List` into a `Map` with `toMap()`?
173. [Intermediate] What happens if `toMap()` encounters duplicate keys without a merge function?
174. [Advanced] What makes a stream operation associative enough to be safely reduced in parallel?
175. [Advanced] Why can side effects inside stream operations cause correctness problems?
176. [Advanced] What happens if you reuse a `Stream` instance after a terminal operation?
177. [Advanced] How can you design a custom collector, and when would you actually need one?
178. [Advanced] What are the differences between `map()`, `flatMap()`, and `mapMulti()` in modern Java?

6. Multithreading & Concurrency

179. [Basic] What is a process and what is a thread?
180. [Basic] What is the difference between concurrency and parallelism?
181. [Basic] How do you create and start a thread in Java?
182. [Basic] What is the difference between start() and run()?
183. [Basic] What is Runnable?
184. [Basic] What is Callable?
185. [Basic] What is Future?
186. [Basic] What is ExecutorService?
187. [Basic] Why is creating threads manually often avoided in server applications?
188. [Basic] What does synchronized mean?
189. [Basic] What is a race condition?
190. [Basic] What is a deadlock?
191. [Intermediate] What is thread safety?
192. [Intermediate] What is the difference between synchronized methods and synchronized blocks?
193. [Intermediate] What object is used as the monitor for a synchronized instance method?
194. [Intermediate] What is the monitor for a static synchronized method?
195. [Intermediate] What is volatile used for?
196. [Intermediate] Does volatile make compound operations such as count++ atomic?
197. [Intermediate] What is atomicity?
198. [Intermediate] What is visibility?
199. [Intermediate] What is ordering in the Java Memory Model?
200. [Intermediate] What is the difference between AtomicInteger and synchronized incrementing?
201. [Intermediate] What is ReentrantLock and when might it be preferable to synchronized?
202. [Intermediate] What is ReadWriteLock?
203. [Intermediate] What is CountdownLatch?
204. [Intermediate] What is CyclicBarrier?
205. [Intermediate] What is Semaphore?
206. [Intermediate] What is a BlockingQueue useful for in producer-consumer systems?
207. [Intermediate] What is ThreadPoolExecutor?
208. [Intermediate] What do corePoolSize, maximumPoolSize, and queue capacity mean?
209. [Intermediate] What can happen if an executor queue is unbounded?
210. [Intermediate] What is a rejected execution policy?
211. [Intermediate] What is CompletableFuture?
212. [Intermediate] What is the difference between thenApply(), thenCompose(), and thenCombine()?
213. [Intermediate] What happens if an exception occurs in a CompletableFuture chain?
214. [Advanced] What is a happens-before relationship?

- 215. [Advanced] Why can double-checked locking be broken without volatile?
- 216. [Advanced] What is safe publication of an object?
- 217. [Advanced] What is thread starvation?
- 218. [Advanced] What is livelock?
- 219. [Advanced] How would you diagnose a deadlock in a running Java application?
- 220. [Advanced] What risks exist when blocking operations are executed on a limited thread pool?
- 221. [Advanced] What is ForkJoinPool and how does work stealing operate at a high level?
- 222. [Advanced] Why can parallel computation combined with shared mutable state produce incorrect results?

7. JVM Internals, Memory Model & Garbage Collection

- 223. [Basic] What are the main responsibilities of the JVM?
- 224. [Basic] What is bytecode?
- 225. [Basic] What is the role of the class loader?
- 226. [Basic] What is the difference between heap and stack memory?
- 227. [Basic] What is Metaspace?
- 228. [Basic] What is a stack frame?
- 229. [Intermediate] What happens during class loading?
- 230. [Intermediate] What are loading, linking, and initialization?
- 231. [Intermediate] What is parent delegation in class loading?
- 232. [Intermediate] What can cause StackOverflowError?
- 233. [Intermediate] What can cause OutOfMemoryError?
- 234. [Intermediate] What is a memory leak in a garbage-collected language?
- 235. [Intermediate] What is garbage collection trying to identify?
- 236. [Intermediate] What is the difference between a strong, soft, weak, and phantom reference?
- 237. [Intermediate] What is stop-the-world?
- 238. [Intermediate] What is the purpose of generational garbage collection?
- 239. [Intermediate] What are young and old generations conceptually?
- 240. [Intermediate] What is an object promotion?
- 241. [Intermediate] What is a GC root?
- 242. [Intermediate] What kinds of objects can become GC roots?
- 243. [Advanced] What is the Java Memory Model and why does it matter for concurrent code?
- 244. [Advanced] What is escape analysis at a high level?
- 245. [Advanced] What is scalar replacement?
- 246. [Advanced] What is the difference between a minor collection and a major/full collection conceptually?
- 247. [Advanced] Compare G1 and a traditional throughput-oriented collector at a high level.
- 248. [Advanced] What JVM metrics would you inspect when investigating frequent GC pauses?
- 249. [Advanced] How would you investigate a suspected memory leak in a Java service?
- 250. [Advanced] Why can a static collection create a memory leak?

251. [Advanced] How can ThreadLocal contribute to memory retention in thread pools?

8. String Handling & Immutability

252. [Basic] Why is String immutable in Java?

253. [Basic] What is the String pool?

254. [Basic] What is the difference between String literals and new String()?

255. [Basic] What is the difference between == and equals() for Strings?

256. [Basic] What does String.intern() do?

257. [Intermediate] Why is StringBuilder preferred for repeated string concatenation in a loop?

258. [Intermediate] What is the difference between StringBuilder and StringBuffer?

259. [Intermediate] How does compile-time string concatenation work?

260. [Intermediate] What happens with "a" + "b" versus variables concatenated at runtime?

261. [Intermediate] Why does String being immutable help with HashMap keys?

262. [Intermediate] How does immutability improve thread safety?

263. [Advanced] What is compact string storage in modern Java, conceptually?

264. [Advanced] What are the trade-offs of interning many dynamically generated strings?

265. [Advanced] How would you design an immutable value object containing a List?

9. Design Patterns & Real Usage

266. [Basic] What problem does the Factory Method pattern solve?

267. [Basic] When would you use a Builder pattern?

268. [Basic] What problem does the Strategy pattern solve?

269. [Basic] What is the Observer pattern and where might it appear in backend systems?

270. [Basic] What is the Adapter pattern?

271. [Basic] What is the Decorator pattern?

272. [Basic] What is the Singleton pattern and what problems can it introduce?

273. [Intermediate] How would you use Strategy to implement multiple discount calculation rules?

274. [Intermediate] How would you use Factory to select a payment provider implementation?

275. [Intermediate] How can dependency injection reduce the need for manually implemented Singleton objects?

276. [Intermediate] What is Template Method and where could it fit in a service workflow?

277. [Intermediate] What is Chain of Responsibility and where could it fit in request validation?

278. [Intermediate] What is Proxy and how is it related to Spring AOP?

279. [Intermediate] What is the difference between Facade and Adapter?

280. [Intermediate] When can a design pattern make code worse rather than better?

281. [Advanced] How would you combine Strategy and Factory without creating a large conditional statement?

282. [Advanced] How would you implement a thread-safe Singleton without unnecessary synchronization overhead?

283. [Advanced] How would you recognize overuse of patterns in a Spring application?

10. SOLID Principles & Clean Code

- 284. [Basic] What does SRP mean?
- 285. [Basic] What does OCP mean?
- 286. [Basic] What does LSP mean?
- 287. [Basic] What does ISP mean?
- 288. [Basic] What does DIP mean?
- 289. [Basic] What is the difference between dependency inversion and dependency injection?
- 290. [Basic] Why should a method generally have one clear responsibility?
- 291. [Intermediate] How can a class violate SRP even when it is not very large?
- 292. [Intermediate] How can inheritance violate LSP?
- 293. [Intermediate] What is a code smell?
- 294. [Intermediate] What is the difference between duplication and necessary abstraction?
- 295. [Intermediate] What is a good reason to extract a method?
- 296. [Intermediate] Why are magic numbers and magic strings problematic?
- 297. [Intermediate] What makes a method difficult to test?
- 298. [Intermediate] How would you refactor a service containing a large if/else or switch over a type?
- 299. [Intermediate] How do meaningful names improve maintainability?
- 300. [Advanced] How would you identify a violation of DIP in a Spring service?
- 301. [Advanced] How can excessive abstraction make a junior-level codebase harder to maintain?
- 302. [Advanced] How would you refactor a 300-line service without changing behavior?
- 303. [Advanced] How would you decide whether a new abstraction is justified?

11. Spring & Spring Boot

- 304. [Basic] What problem does Spring IoC solve?
- 305. [Basic] What is dependency injection?
- 306. [Basic] What is the difference between constructor, setter, and field injection?
- 307. [Basic] Why is constructor injection commonly preferred?
- 308. [Basic] What is a Spring bean?
- 309. [Basic] What is the difference between @Component, @Service, @Repository, and @Controller?
- 310. [Basic] What does @RestController mean?
- 311. [Basic] What does @SpringBootApplication combine?
- 312. [Basic] What is component scanning?
- 313. [Basic] What is auto-configuration in Spring Boot?
- 314. [Basic] What is application.properties versus application.yml?
- 315. [Basic] What are Spring profiles?
- 316. [Basic] What is @Configuration?

- 317. [Basic] What is @Bean?
- 318. [Intermediate] What are common Spring bean scopes?
- 319. [Intermediate] What is the default bean scope?
- 320. [Intermediate] What is the Spring bean lifecycle?
- 321. [Intermediate] What are @PostConstruct and @PreDestroy used for?
- 322. [Intermediate] What happens when two beans of the same type exist and one is injected?
- 323. [Intermediate] How do @Primary and @Qualifier differ?
- 324. [Intermediate] How does Spring resolve circular dependencies, and why should you avoid designing around them?
- 325. [Intermediate] What is AOP and what problems is it good at solving?
- 326. [Intermediate] What is a Spring proxy?
- 327. [Intermediate] What is the difference between JDK dynamic proxies and class-based proxies?
- 328. [Intermediate] What is @Transactional and where should transaction boundaries normally live?
- 329. [Intermediate] What does propagation mean in Spring transactions?
- 330. [Intermediate] What is isolation in a Spring transaction?
- 331. [Intermediate] What is rollback behavior for checked versus unchecked exceptions by default?
- 332. [Intermediate] What is @ControllerAdvice used for?
- 333. [Intermediate] How would you implement consistent REST error responses in Spring Boot?
- 334. [Intermediate] What is Spring Boot Actuator?
- 335. [Intermediate] Which Actuator endpoints are commonly useful for diagnosing a service?
- 336. [Intermediate] How do externalized configuration and environment variables work in Spring Boot?
- 337. [Intermediate] What is @ConfigurationProperties and when is it preferable to many @Value fields?
- 338. [Advanced] Why can calling a @Transactional method from another method in the same class bypass transaction advice?
- 339. [Advanced] What happens when a transactional method is called through a Spring proxy?
- 340. [Advanced] What is the self-invocation problem in Spring AOP?
- 341. [Advanced] How would you troubleshoot a bean that Spring says it cannot find?
- 342. [Advanced] How would you diagnose multiple-bean injection errors?
- 343. [Advanced] What is the difference between filter, interceptor, and AOP advice in a Spring web application?
- 344. [Advanced] What is the difference between Spring MVC and Spring WebFlux at a high level?
- 345. [Advanced] What does @Async require to work through Spring's proxy model?
- 346. [Advanced] How would you safely expose Actuator endpoints in a production-like environment?

12. Spring Data JPA & Hibernate

- 347. [Basic] What is JPA?
- 348. [Basic] What is Hibernate?
- 349. [Basic] What is the difference between JPA and Hibernate?
- 350. [Basic] What is an Entity?

- 351. [Basic] What is the purpose of @Id?
- 352. [Basic] What is the difference between @GeneratedValue strategies?
- 353. [Basic] What is the difference between one-to-one, one-to-many, many-to-one, and many-to-many mappings?
- 354. [Basic] What is lazy loading?
- 355. [Basic] What is eager loading?
- 356. [Intermediate] What is the persistence context?
- 357. [Intermediate] What is the first-level cache in Hibernate?
- 358. [Intermediate] What is dirty checking?
- 359. [Intermediate] What happens when an entity is modified inside a transaction without calling save()?
- 360. [Intermediate] What is the difference between transient, managed, detached, and removed entity states?
- 361. [Intermediate] What is the N+1 query problem?
- 362. [Intermediate] How can JOIN FETCH help with N+1?
- 363. [Intermediate] What is EntityGraph and when can it help?
- 364. [Intermediate] What is the difference between CascadeType.REMOVE and orphanRemoval?
- 365. [Intermediate] What are the risks of using CascadeType.ALL indiscriminately?
- 366. [Intermediate] Why can bidirectional relationships cause JSON serialization problems?
- 367. [Intermediate] How would you prevent infinite recursion when serializing JPA relationships?
- 368. [Intermediate] What is the difference between save(), saveAndFlush(), and flush()?
- 369. [Intermediate] What is optimistic locking?
- 370. [Intermediate] What does @Version do?
- 371. [Intermediate] What is pessimistic locking?
- 372. [Intermediate] When would you use a DTO instead of returning an entity directly from a REST controller?
- 373. [Intermediate] What is the difference between JPQL and native SQL queries?
- 374. [Intermediate] What is a derived query method in Spring Data JPA?
- 375. [Advanced] Why can accessing a lazy relationship outside a transaction cause LazyInitializationException?
- 376. [Advanced] How would you diagnose an N+1 problem from application logs or SQL output?
- 377. [Advanced] Why can EAGER fetching fail to solve N+1 problems?
- 378. [Advanced] How can pagination interact badly with collection fetch joins?
- 379. [Advanced] What is the second-level cache in Hibernate?
- 380. [Advanced] What kinds of data are poor candidates for a second-level cache?
- 381. [Advanced] What is the difference between persist(), merge(), and save-like repository operations conceptually?
- 382. [Advanced] What happens when merge() receives a detached entity?
- 383. [Advanced] How can a large persistence context affect memory usage?

13. REST API Design & Best Practices

- 384. [Basic] What is REST?
- 385. [Basic] What are the main HTTP methods and their typical semantics?
- 386. [Basic] What is the difference between PUT and PATCH?

- 387. [Basic] What is the difference between POST and PUT regarding idempotency?
- 388. [Basic] What are common HTTP status codes for successful, client-error, and server-error responses?
- 389. [Basic] What is content negotiation?
- 390. [Basic] What is JSON serialization?
- 391. [Intermediate] How would you design endpoints for users and their orders?
- 392. [Intermediate] What does idempotency mean for an API?
- 393. [Intermediate] How would you make a payment-like POST operation safely retryable?
- 394. [Intermediate] Where should validation happen in a Spring REST API?
- 395. [Intermediate] What should an API return when a requested resource does not exist?
- 396. [Intermediate] What should an API return for invalid request data?
- 397. [Intermediate] How would you represent validation errors consistently?
- 398. [Intermediate] What is pagination and what parameters would you expose?
- 399. [Intermediate] What is the difference between offset pagination and cursor-based pagination at a high level?
- 400. [Intermediate] What is API versioning and what are common approaches?
- 401. [Intermediate] What is HATEOAS and when would you consider it?
- 402. [Intermediate] What is CORS?
- 403. [Intermediate] What is the difference between authentication and authorization?
- 404. [Advanced] How would you prevent duplicate resource creation when clients retry a request?
- 405. [Advanced] How would you design an API for partial updates without accidentally overwriting fields?
- 406. [Advanced] How would you handle backward compatibility when adding fields to a public API?
- 407. [Advanced] What information should not be exposed in REST error messages?
- 408. [Advanced] How would you structure a REST API that must support filtering, sorting, and pagination?

14. Databases & SQL

- 409. [Basic] What is a primary key?
- 410. [Basic] What is a foreign key?
- 411. [Basic] What is the difference between WHERE and HAVING?
- 412. [Basic] What is the difference between INNER JOIN and LEFT JOIN?
- 413. [Basic] What is a self join?
- 414. [Basic] What is normalization?
- 415. [Basic] What is denormalization?
- 416. [Basic] What is an index?
- 417. [Basic] Why can indexes improve SELECT performance?
- 418. [Basic] Why can indexes slow down INSERT, UPDATE, and DELETE operations?
- 419. [Intermediate] What is a composite index?
- 420. [Intermediate] How does column order matter in a composite index?
- 421. [Intermediate] What is a covering index?
- 422. [Intermediate] What is a database execution plan?

- 423. [Intermediate] How would you investigate a slow SQL query?
- 424. [Intermediate] What is the difference between UNION and UNION ALL?
- 425. [Intermediate] What is the difference between EXISTS and IN?
- 426. [Intermediate] How do NULL values affect comparisons and joins?
- 427. [Intermediate] What are ACID properties?
- 428. [Intermediate] What is a transaction?
- 429. [Intermediate] What are dirty reads, non-repeatable reads, and phantom reads?
- 430. [Intermediate] What are the common transaction isolation levels?
- 431. [Intermediate] What is optimistic concurrency control at the database level?
- 432. [Intermediate] What is a deadlock in a database?
- 433. [Intermediate] How can inconsistent lock ordering contribute to database deadlocks?
- 434. [Intermediate] What is connection pooling?
- 435. [Intermediate] Why can opening a new database connection per request be expensive?
- 436. [Advanced] How would you choose indexes for a query containing WHERE, JOIN, ORDER BY, and LIMIT?
- 437. [Advanced] Why might a database ignore an index even when one exists?
- 438. [Advanced] What is selectivity/cardinality and why does it matter for indexes?
- 439. [Advanced] How can functions applied to indexed columns affect index usage?
- 440. [Advanced] What is the difference between optimistic and pessimistic database locking?
- 441. [Advanced] How would you diagnose a transaction that is unexpectedly holding locks for a long time?
- 442. [Advanced] What SQL query would you write to find duplicate values in a column?
- 443. [Advanced] What SQL query would you write to find the second-highest salary?

15. Testing — JUnit 5, Mockito & Integration

- 444. [Basic] What is unit testing?
- 445. [Basic] What is integration testing?
- 446. [Basic] What is the difference between a unit test and an integration test?
- 447. [Basic] What is JUnit 5?
- 448. [Basic] What are @Test, @BeforeEach, and @AfterEach used for?
- 449. [Basic] What is an assertion?
- 450. [Basic] What is Mockito used for?
- 451. [Basic] What is a mock?
- 452. [Intermediate] What is the difference between mock(), spy(), and real objects in Mockito?
- 453. [Intermediate] What is when(...).thenReturn(...) used for?
- 454. [Intermediate] What is verify() used for?
- 455. [Intermediate] What is ArgumentCaptor used for?
- 456. [Intermediate] What is the difference between @Mock, @InjectMocks, and @Spy?
- 457. [Intermediate] What is a test fixture?
- 458. [Intermediate] How should you test a service that depends on a repository?

- 459. [Intermediate] When should you avoid mocking a dependency?
- 460. [Intermediate] What is a parameterized test in JUnit 5?
- 461. [Intermediate] How do you test exception behavior with JUnit 5?
- 462. [Intermediate] What makes a unit test brittle?
- 463. [Intermediate] What is test isolation?
- 464. [Intermediate] What is `@SpringBootTest` and when would you use it?
- 465. [Intermediate] What is the difference between `@WebMvcTest` and `@SpringBootTest`?
- 466. [Intermediate] How would you test a REST controller without starting the entire application?
- 467. [Intermediate] What is `MockMvc`?
- 468. [Advanced] How would you test transactional business logic without coupling tests to implementation details?
- 469. [Advanced] What is `Testcontainers` and when is it useful?
- 470. [Advanced] How would you test a repository against a real database engine?
- 471. [Advanced] What is the danger of overusing mocks in integration-heavy code?
- 472. [Advanced] How would you test retry logic without making tests slow or flaky?

16. Maven / Gradle, Git & Docker Basics

- 473. [Basic] What is Maven and what problem does it solve?
- 474. [Basic] What is a Maven POM?
- 475. [Basic] What are Maven dependencies and scopes?
- 476. [Basic] What is the Maven lifecycle?
- 477. [Basic] What is the difference between compile, test, package, and install phases?
- 478. [Basic] What is Gradle and how does it differ conceptually from Maven?
- 479. [Basic] What is a Git commit?
- 480. [Basic] What is a Git branch?
- 481. [Basic] What is the difference between merge and rebase?
- 482. [Basic] What is a merge conflict and how do you resolve one?
- 483. [Basic] What is Docker?
- 484. [Basic] What is the difference between an image and a container?
- 485. [Basic] What is a Dockerfile?
- 486. [Basic] What does `docker build` do?
- 487. [Basic] What does `docker run` do?
- 488. [Intermediate] What is a Maven transitive dependency?
- 489. [Intermediate] How do you inspect Maven dependency conflicts?
- 490. [Intermediate] What is `dependencyManagement` in Maven?
- 491. [Intermediate] What is a multi-module Maven project?
- 492. [Intermediate] What is the difference between `git fetch` and `git pull`?
- 493. [Intermediate] What is `git reset` versus `git revert`?
- 494. [Intermediate] What is a detached HEAD?

- 495. [Intermediate] What is a .gitignore file?
- 496. [Intermediate] What is a Docker volume?
- 497. [Intermediate] What is port mapping in Docker?
- 498. [Intermediate] What is the difference between CMD and ENTRYPOINT?
- 499. [Intermediate] Why should configuration and secrets generally not be baked into a Docker image?
- 500. [Intermediate] How would you reduce the size of a Java Docker image?
- 501. [Advanced] How would you diagnose a Maven dependency version conflict?
- 502. [Advanced] When is rebase appropriate and when can it be risky?
- 503. [Advanced] How would you create a multi-stage Docker build for a Spring Boot application?
- 504. [Advanced] How would you investigate a container that starts and immediately exits?

17. Microservices Fundamentals

- 505. [Basic] What is a microservice?
- 506. [Basic] What is the difference between a monolith and a microservices architecture?
- 507. [Basic] What is service discovery?
- 508. [Basic] What is an API gateway?
- 509. [Basic] What is synchronous versus asynchronous communication?
- 510. [Basic] What is a message broker?
- 511. [Basic] What is eventual consistency?
- 512. [Intermediate] Why might a team split a system into multiple services?
- 513. [Intermediate] What problems can microservices introduce compared with a monolith?
- 514. [Intermediate] What is database-per-service and why is it commonly discussed?
- 515. [Intermediate] Why can sharing one database across services create tight coupling?
- 516. [Intermediate] What is a circuit breaker?
- 517. [Intermediate] What is retry, and when can retry make an outage worse?
- 518. [Intermediate] What is timeout configuration and why is it important?
- 519. [Intermediate] What is idempotency in service-to-service communication?
- 520. [Intermediate] What is distributed tracing?
- 521. [Intermediate] What is correlation ID?
- 522. [Intermediate] What is centralized configuration?
- 523. [Intermediate] What is the role of health checks in microservices?
- 524. [Intermediate] What is the difference between liveness and readiness?
- 525. [Advanced] How would you prevent an automatic retry from duplicating a non-idempotent operation?
- 526. [Advanced] How would you design a simple fallback for a dependent service being unavailable?
- 527. [Advanced] Why can synchronous chains across many services create cascading failures?
- 528. [Advanced] What should a strong junior understand about Kafka partitions, consumer groups, and offsets?
- 529. [Advanced] What is the difference between at-most-once, at-least-once, and exactly-once delivery at a high level?

18. Entry-Level System Design

- 530. [Basic] How would you design a simple appointment booking API?
- 531. [Basic] What main components would you identify for a basic library management system?
- 532. [Basic] How would you design a URL shortening service at a small scale?
- 533. [Basic] How would you design a simple file upload service?
- 534. [Intermediate] How would you prevent two users from booking the same appointment concurrently?
- 535. [Intermediate] How would you design the database tables for users, products, and orders?
- 536. [Intermediate] How would you design a notification service that sends email after an event?
- 537. [Intermediate] How would you decide between synchronous REST and asynchronous messaging for a simple workflow?
- 538. [Intermediate] How would you handle retries when one service calls another?
- 539. [Intermediate] Where would you place validation in a simple multi-service system?
- 540. [Intermediate] How would you add caching to a read-heavy endpoint?
- 541. [Intermediate] What would you log for a request that travels through several services?
- 542. [Advanced] How would you design a small order workflow where payment can fail after an order is created?
- 543. [Advanced] How would you prevent duplicate processing of the same message in a consumer?
- 544. [Advanced] How would you design a simple inventory reservation flow that avoids overselling?
- 545. [Advanced] What database indexes would you consider for an appointment-search API, and why?

19. Coding & Problem-Solving — Live Interview

- 546. [Basic] Write a method that checks whether a String is a palindrome.
- 547. [Basic] Write a method that reverses a String without using a library reverse method.
- 548. [Basic] Write a method that counts the frequency of each character in a String.
- 549. [Basic] Write a method that finds the first non-repeated character in a String.
- 550. [Basic] Write a method that removes duplicate integers from a List while preserving insertion order.
- 551. [Basic] Write a method that finds the maximum and minimum values in an integer array.
- 552. [Basic] Write a method that checks whether two Strings are anagrams.
- 553. [Basic] Write a method that counts the number of words in a sentence.
- 554. [Intermediate] Find the first duplicate element in an integer array.
- 555. [Intermediate] Find the missing number from an array containing numbers from 1 to n.
- 556. [Intermediate] Find the second-largest distinct integer in an array.
- 557. [Intermediate] Merge two sorted arrays into one sorted array.
- 558. [Intermediate] Find all pairs in an array whose sum equals a target.
- 559. [Intermediate] Move all zeros in an array to the end while preserving the order of non-zero values.
- 560. [Intermediate] Remove duplicates from a sorted array in place.
- 561. [Intermediate] Find the intersection of two integer arrays.
- 562. [Intermediate] Find the longest substring without repeating characters.

- 563. [Intermediate] Validate whether brackets in a String are balanced.
- 564. [Intermediate] Implement a simple LRU cache using Java collections.
- 565. [Intermediate] Sort a List of objects by multiple fields using Java 8+ APIs.
- 566. [Intermediate] Group a List of employees by department and find the highest salary in each department.
- 567. [Intermediate] Convert a List of objects into a Map while handling duplicate keys.
- 568. [Advanced] Find the top K most frequent elements in an integer array.
- 569. [Advanced] Implement a thread-safe counter and explain the design choices.
- 570. [Advanced] Implement a producer-consumer example using BlockingQueue.
- 571. [Advanced] Detect a cycle in a linked list.
- 572. [Advanced] Given overlapping intervals, merge them into non-overlapping intervals.
- 573. [Advanced] Implement a simple rate limiter for a single application instance.

20. Behavioral & Scenario-Based

- 574. [Basic] Walk me through your most recent Java backend project.
- 575. [Basic] What was your exact responsibility in your most recent project?
- 576. [Basic] Which Java version are you currently using professionally?
- 577. [Basic] Which Spring Boot version have you worked with most recently?
- 578. [Basic] Tell me about a backend feature you implemented from requirements to production.
- 579. [Basic] What part of backend development do you find most challenging?
- 580. [Intermediate] Tell me about a production bug you caused or discovered. How did you investigate it?
- 581. [Intermediate] Tell me about a time you had to understand unfamiliar code quickly.
- 582. [Intermediate] Describe a situation where a database query caused a performance problem.
- 583. [Intermediate] Describe a situation where you disagreed with another developer about an implementation.
- 584. [Intermediate] Tell me about a time you received unclear requirements. What did you do?
- 585. [Intermediate] Tell me about a time you had to deliver under a tight deadline.
- 586. [Intermediate] Describe a refactoring you made and how you verified that behavior did not change.
- 587. [Intermediate] Tell me about a difficult debugging session and the evidence you used to find the root cause.
- 588. [Intermediate] What do you do when you do not know the answer to a technical question during an interview or at work?
- 589. [Intermediate] How do you decide whether a bug should be fixed immediately or scheduled later?
- 590. [Advanced] You inherit a service with poor tests, duplicated code, and frequent production bugs. What would you do first?
- 591. [Advanced] A release is scheduled today and you discover a serious but non-blocking defect. How would you communicate and handle it?
- 592. [Advanced] A senior developer asks you to implement something you believe will create a maintainability problem. How would you raise the concern?
- 593. [Advanced] Your API is intermittently slow in production, but local tests are fast. Walk through your investigation.
- 594. [Advanced] A teammate says your pull request is over-engineered. How would you evaluate and respond to that feedback?

595. [Advanced] You are asked to own a backend feature involving Java, Spring, SQL, and an external API. How would you break down the work?